```
class orderRecieved extends Notification implements
ShouldQueue
{
 use Queueable;
 // Logic
}
```

After your notice has been updated with the **ShouldQueue** interface, you may send it normally. When a class has the **ShouldQueue** interface, Laravel will recognize it and queue up the notification delivery automatically:

```
$user->notify(new orderRecieved($invoice));
```

Every recipient and channel combination will result in the creation of a queued job when queueing notifications. For instance, if your notice includes two channels and three receivers, six jobs will be added to the queue.

## Delaying Notifications

You may link the **delay** method onto your notification instantiation to postpone the notice's delivery:

```
$delay = now()->addMinutes(10);
$user->notify((new orderRecieved($invoice))->delay($delay));
```

The **delay** function allows you to supply an array to set the delay amount for each channels:

```
$user->notify((new orderRecieved($invoice))->delay([
 'mail' => now()->addMinutes(2),
 'sms' => now()->addMinutes(5),
]));
```

As an alternative, the notification class itself may define a **withDelay** method. Channel names and delay values should be returned as an array by the **withDelay** method:

```
public function withDelay(object $notifiable): array
{
 return [
  'mail' => now()->addMinutes(2),
  'sms' => now()->addMinutes(5),
```